

INFRASTRUCTURE DEPLOYMENT WITH TERRAFORM

PHASE 1



DevOps & Azure Provisioning with Terraform + Ansible + Microsoft Entra Connect Sync

Author: Lucas Román Vercellini

Goal: Deploy and configure a hybrid Windows + Azure environment with Domain Controller, identity sync, security hardening, base software, and secure access.

Term	Definition
Terraform	IaC tool that allows you to define, provision, and version infrastructure using declarative syntax (HCL).
Resource Group	Logical container for Azure resources, simplifying management, access, and cost control.
Provider	Terraform plugin that understands how to interact with a specific cloud API (e.g. <code>azurerm</code> for Azure).
State File	File that stores the current infrastructure state, essential for consistent <code>plan</code> and <code>apply</code> .
NSG (Network Security Group)	Firewall rules at the network level to control inbound/outbound traffic to subnets or NICs.
Dynamic IP	IP address assigned automatically by Azure, which may change when the resource is stopped/restarted.

🔧 Project Structure

```
azure-avd-lab/
├─ terraform/
│   ├── main.tf
│   ├── variables.tf
│   ├── outputs.tf
│   ├── providers.tf
│   ├── lab.tfvars
│   └─ bastion.tf (opcional, luego)
├─ ansible/
│   ├── playbook.yml
│   ├── hosts.ini
│   └─ roles/
│       └─ windows_config/
│           ├── tasks/
│           │   ├── main.yml
│           │   └─ hardening.yml
├─ docs/
│   ├── README.md ← documentación principal en inglés
│   └─ screenshots/ ← capturas paso a paso
└─ .gitignore
```

PHASE 1 – Infrastructure Deployment with Terraform

Required Information to Start

Azure Account Data:

1. **Subscription ID**

`az account show --query id -o tsv`

2. **Tenant ID**

`az account show --query tenantId -o tsv`

3. **Resource Group**

Defined via Terraform: `rg-avd-lab`

4. **Region / Location**

Recommended: `East US` (verified for free accounts)

5. **Base Resource Name**

Convention: `avd-lab` (used as prefix for VM, NSG, IP, etc.)

6. **VM Size**

`Standard_B2ms` (2 vCPU, 8 GB RAM) → suitable for Windows & testing

7. **Windows VM Base Image**

- Windows 11 Enterprise multi-session:
 - `publisher = "MicrosoftWindowsDesktop"`
 - `offer = "windows-11"`
 - `sku = "win11-21h2-ent"`
 - `version = "latest"`

8. **Local Administrator Credentials**

- Username: `azureuser`
- Password: defined in `lab.tfvars`
-

Important: Two Separate Environments Are Used

Este laboratorio utiliza **DOS entornos diferentes**:

-  Local Environment (VirtualBox) – Domain Controller (`lab.local`)
Local VM with Windows Server 2022
-  Azure Environment – Target VM deployed with Terraform
VM password is set in `lab.tfvars`

Both environments are synchronized via Microsoft Entra Connect in Phase 3.

Variable Configuration

variables.tf

```
variable "location" {  
  description = "Azure region where resources will be deployed"  
  type       = string  
  default    = "East US"  
}
```

```
variable "resource_group_name" {  
  description = "Name of the resource group"  
  type      = string  
  default    = "rg-avd-lab"  
}
```

```
variable "vm_admin_username" {  
  description = "Admin username for the Windows VM"  
  type      = string  
  default    = "azureuser"  
}
```

```
variable "vm_admin_password" {  
  description = "Admin password for the Windows VM"  
  type      = string  
  sensitive   = true  
}
```

```
variable "vm_name" {  
  description = "Name of the virtual machine"  
  type      = string  
  default    = "vm-avd-lab"  
}
```

```
variable "vm_size" {  
  description = "Azure VM size"  
  type      = string  
  default    = "Standard_B2ms"  
}
```

✅ lab.tfvars

location = "East US"

vm_admin_password = "P@ssw0rd123456!" # Cambiar por uno real y fuerte

✖ Terraform Configuration Files

✅ providers.tf

```
terraform {  
  required_providers {  
    azurerm = {  
      source = "hashicorp/azurerm"  
      version = "~> 3.79.0"  
    }  
  }  
  required_version = ">= 1.4.0"  
}
```

```
provider "azurerm" {  
  features {}  
  skip_provider_registration = true  
}
```

✅ main.tf

```
resource "azurerm_resource_group" "main" {  
  name     = var.resource_group_name  
  location = var.location  
}
```

```
resource "azurerm_virtual_network" "main" {  
  name            = "vnet-avd"  
  address_space   = ["10.0.0.0/16"]  
  location        = azurerm_resource_group.main.location  
  resource_group_name = azurerm_resource_group.main.name  
}
```

```
resource "azurerm_subnet" "main" {  
  name            = "subnet-avd"  
  resource_group_name = azurerm_resource_group.main.name  
  virtual_network_name = azurerm_virtual_network.main.name  
  address_prefixes   = ["10.0.1.0/24"]  
}
```

```
resource "azurerm_network_security_group" "main" {  
  name      = "nsg-avd"  
  location  = azurerm_resource_group.main.location  
  resource_group_name = azurerm_resource_group.main.name  
}
```

```
security_rule {  
  name            = "Allow-RDP-CustomPort"  
  priority        = 1001  
  direction       = "Inbound"  
  access          = "Allow"  
  protocol        = "Tcp"  
  source_port_range = "*"   
  destination_port_range = "49999"  
  source_address_prefix = "Internet"  
  destination_address_prefix = "*"   
}
```

```
security_rule {  
  name      = "Allow-HTTP"  
  priority  = 1002  
  direction = "Inbound"  
  access    = "Allow"
```

```
    protocol          = "Tcp"
    source_port_range  = "*"
    destination_port_range = "80"
    source_address_prefix = "*"
    destination_address_prefix = "*"
  }
}
```

```
resource "azurerm_public_ip" "main" {
  name          = "pip-avd"
  location      = azurerm_resource_group.main.location
  resource_group_name = azurerm_resource_group.main.name
  allocation_method = "Dynamic"
  sku           = "Basic"
}
```

```
resource "azurerm_network_interface" "main" {
  name          = "nic-avd"
  location      = azurerm_resource_group.main.location
  resource_group_name = azurerm_resource_group.main.name
```

```
  ip_configuration {
    name          = "ipconfig1"
    subnet_id     = azurerm_subnet.main.id
    private_ip_address_allocation = "Dynamic"
    public_ip_address_id = azurerm_public_ip.main.id
  }
}
```

```
resource "azurerm_windows_virtual_machine" "main" {
  name          = var.vm_name
  resource_group_name = azurerm_resource_group.main.name
  location      = azurerm_resource_group.main.location
  size          = var.vm_size
  admin_username = var.vm_admin_username
  admin_password = var.vm_admin_password
  network_interface_ids = [azurerm_network_interface.main.id]
```

```
  os_disk {
    caching          = "ReadWrite"
    storage_account_type = "Standard_LRS"
  }
```

```
  source_image_reference {
    publisher = "MicrosoftWindowsDesktop"
    offer     = "windows-11"
    sku       = "win11-21h2-ent"
    version   = "latest"
  }
```

```

}

provision_vm_agent    = true
enable_automatic_updates = true
timezone              = "W. Europe Standard Time"

tags = {
  environment = "lab"
  owner       = "Lucas"
}
}

resource "azurerm_network_interface_security_group_association" "main" {
  network_interface_id = azurerm_network_interface.main.id
  network_security_group_id = azurerm_network_security_group.main.id
}

```

✅ outputs.tf

```

output "resource_group_name" {
  description = "The name of the Resource Group"
  value       = azurerm_resource_group.main.name
}

output "vm_name" {
  description = "The name of the deployed Windows virtual machine"
  value       = azurerm_windows_virtual_machine.main.name
}

output "public_ip_address" {
  description = "The public IP address of the VM (for RDP or Bastion access)"
  value       = azurerm_public_ip.main.ip_address
}

output "rdp_port" {
  description = "Custom RDP port configured in the NSG"
  value       = "49999"
}

```

Initialization & Deployment

1. Verify Azure Connection

az account show

Si la información no es correcta o da error, conectar o reconectar:

az login

o para conectar dispositivo:

az login --use-device-code

2. Register Required Providers(critical)

⚠ Setup the providers manually:

```
az provider register --namespace Microsoft.Network
```

```
az provider register --namespace Microsoft.Compute
```

Verify registration:

```
az provider show --namespace Microsoft.Compute --query "registrationState"
```

```
az provider show --namespace Microsoft.Network --query "registrationState"
```

3. Git Ignore Settings

Create .gitignore in your project directory:

```
.terraform/
```

```
*.tfstate
```

```
*.tfstate.backup
```

```
*.tfvars
```

```
*.log
```

```
*.exe
```

4. Plan and Apply

If provider errors occur:

Clean init if necessary

```
rm -rf .terraform .terraform.lock.hcl
```

```
terraform init
```

```
MINGW64~/c:/Users/Lucas/Documents/Azure-Automations/Azure Provisioning with Terraform and Ansible/terraform
Lucas@ShadowNBK-02 MINGW64 ~/Documents/Azure-Automations (main)
$ cd "Azure Provisioning with Terraform and Ansible/terraform"

Lucas@ShadowNBK-02 MINGW64 ~/Documents/Azure-Automations/Azure Provisioning with Terraform and Ansible/terraform (main)
$ terraform init
Initializing the backend...
Initializing provider plugins...
- Finding hashicorp/azurerm versions matching "~> 3.79.0"...
- Installing hashicorp/azurerm v3.79.0...
- Installed hashicorp/azurerm v3.79.0 (signed by HashiCorp)
Everything up-to-date
[main 65b2b0a] Add folder and structure for AVD provisioning lab with Terraform and Ansible
10 files changed, 0 insertions(+), 0 deletions(-)
create mode 100644 Azure Provisioning with Terraform and Ansible/.gitignore
create mode 100644 Azure Provisioning with Terraform and Ansible/ansible/hosts.ini
create mode 100644 Azure Provisioning with Terraform and Ansible/ansible/playbook.yml
create mode 100644 Azure Provisioning with Terraform and Ansible/ansible/roles/windows_config/tasks/hardening.yml
create mode 100644 Azure Provisioning with Terraform and Ansible/ansible/roles/windows_config/tasks/main.yml
create mode 100644 Azure Provisioning with Terraform and Ansible/docs/README.md
create mode 100644 Azure Provisioning with Terraform and Ansible/terraform/main.tf
create mode 100644 Azure Provisioning with Terraform and Ansible/terraform/outputs.tf
create mode 100644 Azure Provisioning with Terraform and Ansible/terraform/providers.tf
create mode 100644 Azure Provisioning with Terraform and Ansible/terraform/variables.tf
Enumerating objects: 11, done.
Counting objects: 100% (11/11), done.
Delta compression using up to 4 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (10/10), 871 bytes | 290.00 KiB/s, done.
total 10 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/ivercell/Azure-Automations.git
   d970090..65b2b0a  main -> main
Initializing the backend...
Initializing provider plugins...
- Finding hashicorp/azurerm versions matching "~> 3.79.0"...
- Installing hashicorp/azurerm v3.79.0...
- Installed hashicorp/azurerm v3.79.0 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```


Verify configuration:

terraform plan -var-file="lab.tfvars"

```
MINGW64: C:\Users\Lucas\Documents\Azure-Automations\Azure Provisioning with Terraform and Ansible\terraform
+ name = "vm-avd-lab"
+ network_interface_ids = (known after apply)
+ patch_assessment_mode = "ImageDefault"
+ patch_mode = "AutomaticByOS"
+ platform_fault_domain = -1
+ priority = "Regular"
+ private_ip_address = (known after apply)
+ private_ip_addresses = (known after apply)
+ provision_vm_agent = true
+ public_ip_address = (known after apply)
+ public_ip_addresses = (known after apply)
+ resource_group_name = "rg-avd-lab"
+ size = "Standard_B2ms"
+ tags = {
  + "environment" = "lab"
  + "owner" = "Lucas"
}
+ timezone = "W. Europe Standard Time"
+ virtual_machine_id = (known after apply)

+ os_disk {
  + caching = "ReadWrite"
  + disk_size_gb = (known after apply)
  + name = (known after apply)
  + storage_account_type = "Standard_LRS"
  + write_accelerator_enabled = false
}

+ source_image_reference {
  + offer = "windows-11"
  + publisher = "MicrosoftWindowsDesktop"
  + sku = "win11-21h2-ent"
  + version = "latest"
}

+ termination_notification (known after apply)
}

Plan: 8 to add, 0 to change, 0 to destroy.

Changes to Outputs:
+ public_ip_address = (known after apply)
+ rdp_port = "49999"
+ resource_group_name = "rg-avd-lab"
+ vm_name = "vm-avd-lab"

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.
```

If everything is ok, then apply:

terraform apply -var-file="lab.tfvars"

```
Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

azurerm_windows_virtual_machine.main: Creating...
azurerm_windows_virtual_machine.main: Still creating... [00m10s elapsed]
azurerm_windows_virtual_machine.main: Still creating... [00m20s elapsed]
azurerm_windows_virtual_machine.main: Still creating... [00m30s elapsed]
azurerm_windows_virtual_machine.main: Still creating... [00m40s elapsed]
azurerm_windows_virtual_machine.main: Still creating... [00m50s elapsed]
azurerm_windows_virtual_machine.main: Still creating... [01m00s elapsed]
azurerm_windows_virtual_machine.main: Still creating... [01m10s elapsed]
azurerm_windows_virtual_machine.main: Still creating... [01m20s elapsed]
azurerm_windows_virtual_machine.main: Still creating... [01m30s elapsed]
azurerm_windows_virtual_machine.main: Still creating... [01m40s elapsed]
azurerm_windows_virtual_machine.main: Still creating... [01m50s elapsed]
azurerm_windows_virtual_machine.main: Still creating... [02m00s elapsed]
azurerm_windows_virtual_machine.main: Still creating... [02m10s elapsed]
azurerm_windows_virtual_machine.main: Still creating... [02m20s elapsed]
azurerm_windows_virtual_machine.main: Still creating... [02m30s elapsed]
azurerm_windows_virtual_machine.main: Still creating... [02m40s elapsed]
azurerm_windows_virtual_machine.main: Still creating... [02m50s elapsed]
azurerm_windows_virtual_machine.main: Still creating... [03m00s elapsed]
azurerm_windows_virtual_machine.main: Still creating... [03m10s elapsed]
azurerm_windows_virtual_machine.main: Still creating... [03m20s elapsed]
azurerm_windows_virtual_machine.main: Still creating... [03m30s elapsed]
azurerm_windows_virtual_machine.main: Still creating... [03m40s elapsed]
azurerm_windows_virtual_machine.main: Creation complete after 3m42s [id=/subscriptions/46a83ca6-544a-40a1-9c67-5058b6264dfd/resourceGroups/rg-avd-lab/providers/Microsoft.Compute/virtualMachines/vm-avd-lab]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Outputs:
rdp_port = "49999"
resource_group_name = "rg-avd-lab"
vm_name = "vm-avd-lab"
```

✔ Outputs and Final Check

Example after successful apply:

Outputs:

public_ip_address = "20.125.76.22"

rdp_port = "49999"

resource_group_name = "rg-avd-lab"

vm_name = "vm-avd-lab"

📌 Final Lab Overview

- Region: East US
- Subscription ID: <your subscription ID>
- Tenant: Default Directory

Created Resources:

- Resource Group: rg-avd-lab
- Virtual Network: vnet-avd
- Subnet: subnet-avd
- NSG: nsg-avd (with RDP only from port 49999)
- NIC: nic-avd
- Windows VM: vm-avd-lab (dynamic public IP)

✅ Infrastructure provisioned successfully

Resource groups

Default Directory (soporteksircom.onmicrosoft.com)

Create

Manage view

Refresh

Export to CSV

Open query

Assign tags

Group by none

You are viewing a new version of Browse experience. Click here to access the old experience.

Filter for any field...

Subscription equals all

Location equals all

Add filter

Name ↑	Subscription	Location
NetworkWatcherRG	Azure subscription 1	East US
rg-avd-lab	Azure subscription 1	East US
rg-LicenciasEastUS	Azure subscription 1	East US
rg-LicenciasLab	Azure subscription 1	East US

Home > Resource groups >

Resource groups

Default Directory (soporteksircom.onmicrosoft.com)

Create

Manage view

You are viewing a new version of Browse experience. Click here to access the old experience.

Name ↑

NetworkWatcherRG

rg-avd-lab

rg-LicenciasEastUS

rg-LicenciasLab

Overview

Activity log

Access control (IAM)

Tags

Resource visualizer

Events

Settings

Cost Management

Monitoring

Automation

Help

Showing 1 - 4 of 4. Display count: auto

rg-avd-lab

Resource group

How do I troubleshoot issues with this resource group?

How do I monitor this resource group?

What are the best practices for managing this resource group?

Search

Create

Manage view

Delete resource group

Refresh

Export to CSV

Open query

Assign tags

Move

Delete

JSON View

Essentials

Subscription (move) : Azure subscription 1

Deployments : No deployments

Subscription ID : 46a83ca6-544a-40a1-9c67-5058b6264dfd

Location : East US

Tags (edit) : Add tags

Resources

Recommendations

Filter for any field...

Type equals all

Location equals all

Add filter

Showing 1 to 5 of 5 records.

Show hidden types

No grouping

List view

Name ↑↓	Type ↑↓	Location ↑↓
nic-avd	Network Interface	East US
nsg-avd	Network security group	East US
vm-avd-lab	Virtual machine	East US
vm-avd-lab_OsDisk_1_e1cfcafa9b45486ea47e2ffc3871f4a5	Disk	East US
vnet-avd	Virtual network	East US

< Previous

Page 1 of 1

Next >

Give feedback